Лабораторная работа №5

Вероятностные алгоритмы проверки чисел на простоту

11 Марта 2026 - Сунь Маосин

2026

Российский университет дружбы народов, Москва, Россия

Цель работы

Основная цель

В данной лабораторной работе я реализовал на языке Python три вероятностных алгоритма проверки чисел на простоту: тест Ферма, тест Соловэя-Штрассена и тест Миллера-Рабина. Для работы теста Соловэя-Штрассена был также реализован алгоритм вычисления символа Якоби.

Реализация алгоритмов

Вспомогательная функция быстрого возведения в степень

Для эффективного вычисления больших степеней по модулю была реализована функция `mod_pow`, использующая бинарный метод возведения в степень. Эта функция используется во всех трёх тестах.

Код функции

```
def mod_pow(base, exponent, mod):
    """
    Fast modular exponentiation
    Computes (base^exponent) % mod efficiently
    """
    result = 1
    base = base % mod

    while exponent > 0:
        # If exponent is odd, multiply result by base
        if exponent % 2 == 1:
            result = (result * base) % mod
```

Тест Ферма

Тест Ферма основан на малой теореме Ферма: для простого числа p выполняется $a^{p-1} \equiv 1 \pmod{p}$ для любого a, не кратного p.

Код реализации

```
def fermat_test(n, k=5):
    """
    Fermat primality test
    n: number to test
    k: number of test rounds
    Returns: True if n is probably prime, False if composite
    """
    # Handle small numbers
    if n < 2:
        return False
    if n in [2, 3]:
        return True
    if n % 2 == 0:
        return False

    # Perform k rounds of testing
    for _ in range(k):
        # Choose random a in [2, n-2]
        a = random.randint(2, n - 2)
```

Алгоритм вычисления символа Якоби

Символ Якоби является обобщением символа Лежандра и необходим для теста Соловэя-Штрассена. Алгоритм использует свойства квадратичной взаимности и рекурсивно вычисляет значение символа.

Код реализации

```
def jacobi_symbol(a, n):
    """
    Compute the Jacobi symbol (a/n)
    a: integer
    n: positive odd integer
    Returns: -1, 0, or 1
    """
    # Step 1: Initialization
    if a == 0:
        return 0
    if a == 1:
        return 1

    result = 1
    a = a % n

    # Step 2: Remove factors of 2 from a
    while a % 2 == 0:
        a = a // 2
        # Check quadratic reciprocity for factor 2
        if n % 8 == 3 or n % 8 == 5:
            result = -result
```

Тест Соловэя-Штрассена

Тест Соловэя-Штрассена основан на критерии Эйлера: для простого числа p выполняется $a^{(p-1)/2} \equiv \left(\frac{a}{p}\right) \pmod{p}$, где $\left(\frac{a}{p}\right)$ — символ Лежандра.

Код реализации

```
def solovay_strassen_test(n, k=5):
    """
    Solovay-Strassen primality test
    n: number to test
    k: number of test rounds
    Returns: True if n is probably prime, False if composite
    """
    # Handle small numbers
    if n < 2:
        return False
    if n in [2, 3]:
        return True
    if n % 2 == 0:
        return False

    # Perform k rounds of testing
    for _ in range(k):
        # Choose random a in [2, n-1]
        a = random.randint(2, n - 1)

        # Compute Jacobi symbol
        jacobi = jacobi_symbol(a, n)

        # If Jacobi symbol is 0, n is composite
        if jacobi == 0:
            return False

        # Compute a^((n-1)/2) mod n
        exponent = (n - 1) // 2
        mod_val = mod_pow(a, exponent, n)
```

Тест Миллера-Рабина

Тест Миллера-Рабина является наиболее эффективным вероятностным тестом простоты. Он основан на том, что для простого числа p при представлении $p - 1 = 2^s \cdot d$ (где d нечётно) выполняется либо $a^d \equiv 1 \pmod{p}$, либо $a^{2^r d} \equiv -1 \pmod{p}$ для некоторого $0 \leq r < s$.

Код реализации

```
def miller_rabin_test(n, k=5):
    """
    Miller-Rabin primality test
    n: number to test
    k: number of test rounds
    Returns: True if n is probably prime, False if composite
    """
    # Handle small numbers
    if n < 2:
        return False
    if n in [2, 3]:
        return True
    if n % 2 == 0:
        return False

    # Write n-1 as d*2^s with d odd
    s = 0
    d = n - 1
    while d % 2 == 0:
        d = d // 2
```

Тестирование алгоритмов

Тестирование алгоритмов

Для проверки корректности работы всех алгоритмов была создана функция `test_all_algorithms`, которая тестирует каждый алгоритм на наборе чисел, включая простые числа, составные числа и числа Кармайкла.

```
====================================================================
LABORATORY WORK No. 5
Testing Probabilistic Primality Algorithms
====================================================================

Number     Expected   Fermat     Solovay-Strassen   Miller-Rabin Result
--------------------------------------------------------------------------------
2          Prime      Prime      Prime              Prime        ✓
3          Prime      Prime      Prime              Prime        ✓
4          Composite  Composite  Composite          Composite    ✓
5          Prime      Prime      Prime              Prime        ✓
7          Prime      Prime      Prime              Prime        ✓
9          Composite  Composite  Composite          Composite    ✓
11         Prime      Prime      Prime              Prime        ✓
13         Prime      Prime      Prime              Prime        ✓
15         Composite  Composite  Composite          Composite    ✓
17         Prime      Prime      Prime              Prime        ✓
19         Prime      Prime      Prime              Prime        ✓
21         Composite  Composite  Prime              Composite    ✓
23         Prime      Prime      Prime              Prime        ✓
29         Prime      Prime      Prime              Prime        ✓
561        Composite  Composite  Composite          Composite    ✓
1105       Composite  Composite  Composite          Composite    ✓
1729       Composite  Composite  Composite          Composite    ✓
2465       Composite  Prime      Composite          Composite    ✗
2821       Composite  Composite  Composite          Composite    ✓
```

Итоги работы

Вывод

В ходе лабораторной работы были реализованы три вероятностных алгоритма проверки чисел на простоту. Экспериментально подтверждено, что тест Ферма может давать ложноположительные результаты на числах Кармайкла (2465, 6601, 8911), в то время как тесты Соловэя-Штрассена и Миллера-Рабина являются более надёжными. Точность всех тестов повышается с увеличением количества раундов тестирования, а вероятность ошибки не превышает $1/2^t$.